



Simulation d'un panache de fumée en 2D

Analyse des fichiers C++ / Qt fournis

EI4 2025-2026

Kim Bumjun, ZHANG

Sommaire

1. Introduction
2. Objectif du programme
3. Organisation générale du code
4. Modélisation de la fumée
5. Prise en compte du vent
6. Génération de la heatmap et affichage Qt
7. Analyse détaillée des principaux fichiers
8. Limites actuelles et pistes d'amélioration
9. Conclusion

1. Introduction

Le programme étudié vise à représenter la fumée émise par une cheminée d'usine sur une zone rectangulaire discrétisée. La fumée n'est pas stockée sous forme d'objets graphiques indépendants ; elle est modélisée par un champ scalaire q , c'est-à-dire une valeur de concentration associée à chaque point de la grille.

Le fonctionnement général peut se résumer en trois étapes : créer la grille, injecter de la fumée sous forme de gaussienne 2D, puis transformer les valeurs numériques du champ en image colorée affichée dans une interface Qt.

Par rapport au rapport modèle, la version ici analysée est plus orientée programmation : elle montre clairement comment une formule mathématique devient un tableau de données, puis une image, puis un affichage interactif.

2. Objectif du programme

L'objectif principal est de simuler et visualiser un panache simple. Le code disponible gère surtout : la description d'une source de fumée, le stockage du champ de concentration, l'émission initiale ou continue, la décomposition du vent en composantes horizontale et verticale, et enfin le rendu graphique de la carte de densité.

Le cœur scientifique du code fourni se situe dans la classe `Smoke`, tandis que le cœur visuel se situe dans `HeatMapRenderer` et `HeatMapWidget`. La classe `Wind` joue un rôle de liaison entre une vitesse en norme/angle et les composantes u et w utilisées pour l'affichage.

3. Organisation générale du code

Fichier	Rôle	Idée principale
<code>chimney.h</code>	Source	Paramètres physiques de la cheminée : position, étalement, amplitude, mode continu, débit.
<code>smoke.h</code> / <code>smoke.cpp</code>	Modèle numérique	Crée la grille, stocke q dans un vecteur 1D, ajoute la gaussienne, remet à zéro, gère l'émission continue.
<code>wind.h</code> / <code>wind.cpp</code>	Vent	Convertit une vitesse et un angle en composantes horizontale u et verticale w .
<code>heatmaprenderer.h</code> / <code>.cpp</code>	Rendu image	Normalise le champ q et le transforme en couleurs pour fabriquer une <code>QImage</code> .

heatmapwidget.h / .cpp	Affichage Qt	Dessine la heatmap, les axes, la barre de couleur, les cheminées et la flèche du vent.
main.cpp	Lancement	Démarre QApplication et affiche la fenêtre principale.

Cette architecture sépare bien la partie calcul et la partie interface. Cela rend le projet plus lisible : on peut modifier la formule de fumée sans changer l’affichage, ou changer la palette de couleurs sans toucher au modèle physique.

4. Modélisation de la fumée

La fumée est stockée dans un vecteur `q_`. Même si le problème est en 2D, les données sont placées en mémoire dans un tableau 1D. La conversion entre les coordonnées de grille (*i*, *j*) et l’indice mémoire est réalisée par la formule `idx(i,j) = i * ny_ + j`.

L’initialisation de la grille se fait dans `initialize(nx1, ny1, l1, h1)`. Cette fonction fixe le nombre de points selon *x* et *y*, calcule les pas spatiaux *dx* et *dy*, puis remplit `q_` avec des zéros.

Le choix d’un vecteur 1D simplifie le stockage mémoire et le parcours des données. Mathématiquement, cela ne change pas le modèle ; c’est seulement une manière efficace de ranger une matrice en mémoire.

L’ajout de fumée se fait dans `addGaussian(const Chimney& c)`. Pour chaque point de la grille, le programme calcule la position physique $x = i * dx_$ et $y = j * dy_$, puis ajoute une contribution gaussienne 2D centrée en (*x0*, *y0*).

La formule programmée correspond à un produit de deux gaussiennes 1D : une selon *x* et une selon *y*. Les paramètres `sigmaX` et `sigmaY` contrôlent l’étalement horizontal et vertical du panache, tandis que `amplitude` contrôle la quantité injectée.

$$q(x,y) += A \cdot (1/(\sigma_x\sqrt{2\pi})) \cdot (1/(\sigma_y\sqrt{2\pi})) \cdot \exp(-(x-x_0)^2/(2\sigma_x^2)) \cdot \exp(-(y-y_0)^2/(2\sigma_y^2))$$

Ainsi, plus un point est proche du centre de la cheminée, plus la valeur ajoutée est grande. À l’inverse, plus on s’éloigne du centre, plus la concentration décroît rapidement.

La fonction `reset()` efface le champ en remettant toutes les valeurs à 0.0. Elle sert à repartir d’un état vide avant une nouvelle simulation ou une nouvelle émission.

5. Prise en compte du vent

Le vent est représenté par la classe `Wind`. L’utilisateur fournit une norme `speed` et un angle `angleDeg`. La méthode `set()` convertit ensuite ces données en composantes cartésiennes : `horizontal = speed * cos(rad)` et `vertical = speed * sin(rad)`.

$$u = \text{vitesse horizontale}, \quad w = \text{vitesse verticale}$$

Les accesseurs `u()`, `w()`, `speed()` et `angleDeg()` permettent de récupérer ces valeurs sans exposer directement les attributs internes.

Dans les fichiers fournis, le vent sert surtout à l'affichage de la flèche dans le widget. Le transport complet de la fumée par advection n'apparaît pas encore dans les fichiers reçus. Autrement dit, le code fourni modélise bien l'émission et la visualisation, mais pas encore un schéma numérique complet de déplacement temporel du champ.

6. Génération de la heatmap et affichage Qt

Une fois que les concentrations ont été calculées, il faut les transformer en image. C'est le rôle de `HeatMapRenderer`.

La méthode `render(q, nx, ny, autoScale)` commence par chercher la valeur maximale du champ si `autoScale` vaut `true`. Les valeurs sont ensuite normalisées entre 0 et 1. Cette normalisation est essentielle : elle permet de convertir des concentrations physiques en niveaux de couleur.

La fonction `mapToColor(v01)` associe ensuite chaque valeur normalisée à une couleur par interpolation entre plusieurs points de palette. La palette retenue va du bleu sombre vers le jaune en passant par le violet, le magenta et l'orange. Le résultat final est une image `QImage`.

Dans `HeatMapWidget`, la méthode `paintEvent()` dessine cette image dans une zone de tracé, puis ajoute les éléments de lecture : axes X et Y, graduations, rectangle de cadre, barre de couleur, étiquette « Density », position des cheminées et flèche représentant le vent.

La barre de couleur est cohérente avec l'image grâce à la fonction `plasmaColor()` définie dans le widget. Les bornes numériques de la légende sont fournies par `setColorScale(vmin, vmax)`.

Le widget effectue aussi une transformation géométrique simple : la coordonnée physique `x` est envoyée vers la droite de l'écran, tandis que `y` est inversée pour être compatible avec les coordonnées graphiques Qt, où l'origine se situe en haut à gauche.

Élément dessiné	Fonction dans l'interface
Heatmap	Montre la densité de fumée sous forme de couleurs.
Axe X / Axe Y	Permettent de lire la position spatiale.
Barre de couleur	Fait le lien entre la couleur et la valeur de densité.
Ellipse des cheminées	Montre la position de la ou des sources.
Flèche du vent	Indique la direction et le sens du vent.

7. Analyse détaillée des principaux fichiers

7.1 Fichier chimney.h

La structure Chimney regroupe les paramètres d'une source ponctuelle. x0 et y0 donnent la position. sigmaX et sigmaY définissent la largeur du nuage. amplitude donne la quantité de fumée injectée. continuous indique si la cheminée émet dans le temps, et rate donne le débit d'émission.

7.2 Fichiers smoke.h et smoke.cpp

La classe Smoke est le cœur du modèle. Elle contient la grille, les pas spatiaux, le champ de concentration, et une liste de cheminées.

Le choix `vector<Chimney> chimneys_` montre que le code est déjà pensé pour gérer plusieurs sources, même si une seule cheminée est utilisée dans beaucoup de cas de test.

La méthode `emitContinuous(double dt)` parcourt toutes les cheminées. Si une cheminée est en mode continu et possède un débit positif, le programme crée une copie locale nommée `injected`, remplace son amplitude par `rate * dt`, puis appelle `addGaussian(injected)`.

Cela signifie qu'à chaque pas de temps, une nouvelle petite quantité de fumée est ajoutée au champ. Plus `dt` est grand, plus la quantité injectée pendant ce pas est importante.

7.3 Fichiers wind.h et wind.cpp

La classe Wind est volontairement simple. Elle sert surtout d'outil de conversion entre une description polaire du vent et une description cartésienne exploitable par le reste du programme.

7.4 Fichiers heatmaprender.h et heatmaprender.cpp

Ces fichiers traduisent les données numériques en image. Ils mémorisent aussi `lastMin_` et `lastMax_`, ce qui permet ensuite d'afficher une légende cohérente avec les dernières données rendues.

7.5 Fichiers heatmapwidget.h et heatmapwidget.cpp

Ces fichiers gèrent l'aspect visuel. Le widget stocke l'image courante, les cheminées, le vent et les bornes de l'échelle. À chaque modification, `update()` demande à Qt de redessiner le widget.

On remarque que le code contient des marges dédiées pour préserver la lisibilité : place pour les axes, pour la barre de couleur et pour la zone de dessin principale.

7.6 Fichier main.cpp

Le main est très court. Il crée `QApplication`, instancie `MainWindow`, affiche la fenêtre puis lance la boucle d'événements avec `a.exec()`. Cela confirme que le projet est bien une application graphique Qt Widgets.

8. Limites actuelles et pistes d'amélioration

À partir des seuls fichiers fournis, plusieurs limites apparaissent.

- Le déplacement temporel complet du panache par équation d'advection n'est pas visible dans les fichiers reçus.

- Le modèle ne prend pas encore en compte la diffusion, la turbulence, la température ou les obstacles.
- Le champ est affiché, mais aucune échelle physique détaillée des axes n'est encore reliée explicitement à des unités d'ingénierie dans le widget.
- La palette est fixe ; un choix de palettes multiples pourrait améliorer la lecture.

Des améliorations possibles seraient : ajouter un vrai schéma d'advection pas à pas, gérer plusieurs types de sources, rendre le vent variable dans l'espace et le temps, permettre le zoom dans l'interface, ou exporter des images et des données de simulation.

9. Conclusion

Le code fourni met déjà en place une base propre pour un projet de simulation de fumée. La modélisation de la source est claire grâce à la gaussienne 2D, le stockage mémoire est simple et efficace grâce au vecteur 1D, et l'interface Qt offre une représentation visuelle immédiatement exploitable.

Même si tous les éléments d'une simulation physique complète ne sont pas encore présents dans les fichiers reçus, l'architecture actuelle est cohérente. Elle permet de comprendre comment on passe d'une idée physique simple à une structure de données, puis à un rendu graphique lisible.

Ce rapport peut donc servir de base de compte rendu ou de soutenance, en montrant à la fois le sens physique du projet et le rôle précis de chaque morceau de code.

Annexe - Correspondance entre idées physiques et code

Idée	Implémentation dans le code
Zone discrétisée	<code>initialize()</code> calcule <code>nx_</code> , <code>ny_</code> , <code>dx_</code> , <code>dy_</code> et crée <code>q_</code> .
Concentration locale	<code>q_[idx(i,j)]</code> stocke la valeur au point <code>(i,j)</code> .
Source ponctuelle	<code>Chimney</code> contient <code>x0</code> , <code>y0</code> , <code>sigmaX</code> , <code>sigmaY</code> et <code>amplitude</code> .
Injection initiale	<code>addGaussian()</code> ajoute une gaussienne 2D sur toute la grille.
Émission continue	<code>emitContinuous(dt)</code> ajoute <code>rate * dt</code> à chaque pas.
Vent	<code>Wind::set()</code> calcule <code>u</code> et <code>w</code> à partir de <code>speed</code> et <code>angleDeg</code> .
Visualisation	<code>HeatMapRenderer::render()</code> transforme <code>q</code> en <code>QImage</code> .